

```

/*
=====
Contrôle de la qualité des données
Objectif :
Vérifier que les données importées depuis les fichiers
CSV ont été correctement chargées dans PostgreSQL
avant de commencer les analyses.
Les contrôles réalisés permettent de :
- Vérifier le volume de données importées
- Détecter d'éventuels doublons sur les identifiants
- Contrôler la présence de valeurs nulles
- S'assurer de la cohérence des données
- Valider les clés utilisées pour les futures jointures
Cette étape est indispensable afin de garantir la
fiabilité des analyses et des indicateurs métier qui
seront produits par la suite.
=====
Vérification du nombre d'enregistrements présents
dans chaque table afin de confirmer que l'import
s'est déroulé correctement.

SELECT COUNT(*)
FROM customers;

SELECT COUNT(*)
FROM products;

SELECT COUNT(*)
FROM orders;

SELECT COUNT(*)
FROM order_items;

SELECT * FROM customers LIMIT 10;

SELECT * FROM products LIMIT 10;

SELECT * FROM orders LIMIT 10;

SELECT * FROM order_items LIMIT 10;
=====
Recherche de doublons sur les identifiants principaux.
Les clés primaires doivent être uniques. La présence
de doublons pourrait entraîner des erreurs dans les
analyses et les jointures entre tables.

SELECT customer_id, COUNT(*) FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1;

SELECT product_id, COUNT(*) FROM products
GROUP BY product_id
HAVING COUNT(*) > 1

```

```
SELECT order_id, COUNT(*) FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1
```

```
SELECT order_id,
       order_item_id,
       COUNT(*)
FROM order_items
GROUP BY order_id, order_item_id
HAVING COUNT(*) > 1;
```

=====

Contrôle des valeurs manquantes

Objectif :

Identifier les colonnes contenant des valeurs NULL

susceptibles d'impacter les analyses futures.

Une attention particulière est portée aux identifiants,
aux dates d'achat et aux montants utilisés dans les KPI.

```
SELECT
SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS
null_customer_id,
SUM(CASE WHEN customer_unique_id IS NULL THEN 1 ELSE 0 END) AS
null_customer_unique_id,
SUM(CASE WHEN customer_zip_code_prefix IS NULL THEN 1 ELSE 0 END) AS
null_zip_code,
SUM(CASE WHEN customer_city IS NULL THEN 1 ELSE 0 END) AS
null_customer_city,
SUM(CASE WHEN customer_state IS NULL THEN 1 ELSE 0 END) AS
null_customer_state
FROM customers;
```

```
SELECT
SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS null_order_id,
SUM(CASE WHEN order_item_id IS NULL THEN 1 ELSE 0 END) AS
null_order_item,
SUM(CASE WHEN product_id IS NULL THEN 1 ELSE 0 END) AS
null_product_id,
SUM(CASE WHEN seller_id IS NULL THEN 1 ELSE 0 END) AS
null_seller_id,
SUM(CASE WHEN shipping_limit_date IS NULL THEN 1 ELSE 0 END) AS
null_shipping_limit_date,
SUM(CASE WHEN price IS NULL THEN 1 ELSE 0 END) AS null_price,
SUM(CASE WHEN freight_value IS NULL THEN 1 ELSE 0 END) AS
null_freight_value
FROM order_items;
```

```
SELECT
SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS null_order_id,
SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS
null_customer_id,
SUM(CASE WHEN order_status IS NULL THEN 1 ELSE 0 END) AS
null_status,
```

```

SUM(CASE WHEN order_purchase_timestamp IS NULL THEN 1 ELSE 0 END)
AS null_purchase_timestamp,
SUM(CASE WHEN order_approved_at IS NULL THEN 1 ELSE 0 END) AS
null_approved_at,
SUM(CASE WHEN order_delivered_carrier_date IS NULL THEN 1 ELSE 0
END) AS null_delivered_carrier_date,
SUM(CASE WHEN order_delivered_customer_date IS NULL THEN 1 ELSE 0
END) AS null_delivered_customer_date,
SUM(CASE WHEN order_estimated_delivery_date IS NULL THEN 1 ELSE 0
END) AS null_estimated_delivery_date
FROM orders;

```

```

SELECT
SUM(CASE WHEN product_id IS NULL THEN 1 ELSE 0 END) AS
null_product_id,
SUM(CASE WHEN product_category_name IS NULL THEN 1 ELSE 0 END) AS
null_category_name
FROM products;

```

=====

Contrôle de cohérence métier

Objectif :

Vérifier la cohérence des données d'un point de vue métier avant de démarrer les analyses.

Les contrôles réalisés portent uniquement sur les éléments ayant un impact direct sur les indicateurs de performance et les futures visualisations.

--Vérification des commandes sans client associé

```

SELECT o.order_id,
       o.customer_id
FROM orders o
LEFT JOIN customers c
      ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;

```

--Vérification des lignes de commande sans produit associé

```

SELECT oi.order_item_id,
       oi.product_id
FROM order_items oi
LEFT JOIN products p
      ON oi.product_id = p.product_id
WHERE p.product_id IS NULL

```

-- Vérification des prix incohérents

```
SELECT order_item_id, price
FROM order_items
WHERE price <= 0;
```

```
SELECT MIN(price)
FROM order_items;
```

```
SELECT MAX(price)
FROM order_items;
```

```
SELECT AVG(price)
FROM order_items;
```

-- Vérification de la cohérence chronologique des dates

```
SELECT order_id, order_purchase_timestamp, order_approved_at
FROM orders
WHERE order_approved_at < order_purchase_timestamp
```

```
SELECT
order_id,order_purchase_timestamp,order_delivered_customer_date
FROM orders
WHERE order_delivered_customer_date < order_purchase_timestamp
```

-- Analyse des statuts de commande

```
SELECT DISTINCT(order_status)
FROM orders
```